

AcquirerX Deep-Dive 4/10 — Merchant Service

Service: `merchant-service` **Port:** `9091` **Database:** `acquirerx_merchant` **Role:** Owns the merchant lifecycle — onboarding, KYC, pricing, settlement profiles, and physical store records

1. What It Is

In an acquiring system, a **merchant** is a business that accepts card payments. Examples in AcquirerX: TechMart Retail, FoodieHub, FashionForward, LuxeJewels.

Merchant-service owns everything about merchants and their physical retail locations:

- **Merchant** — the legal entity (e.g., "TechMart Retail Pvt Ltd")
- **Store** — a physical location operated by a merchant (e.g., "TechMart - MG Road")
- **MerchantKYC** — identity documents proving the merchant is real
- **PricingModel** — what fees this merchant pays
- **SettlementProfile** — how and when this merchant gets paid

Without a complete record across these five entities, a merchant cannot be activated, terminals cannot be assigned, and transactions cannot be processed.

2. Why It's a Separate Service

In a monolith, all of this would be packages. Splitting into a microservice gives:

- **Independent scaling** — onboarding is bursty (new merchants in waves) and shouldn't affect transaction throughput
 - **Domain isolation** — merchant data has its own lifecycle (KYC review takes days), separate from real-time transaction flow
 - **Clean ownership** — merchant data has one writer (this service); other services read via Feign
 - **Compliance boundary** — KYC documents are sensitive; isolating them simplifies access control
-

3. Tech Stack

- Spring Boot 3.x
- Spring Data JPA + MySQL
- Lombok (`@RequiredArgsConstructor`, `@Slf4j`, `@Getter`, `@Setter`)
- Spring Cloud Eureka client (registers itself)
- Bean Validation (`@NotNull`, `@Size`, `@Email` on DTOs)

No Spring Security in this service — JWT is validated at the gateway, downstream services trust the forwarded headers.

No Feign clients here — merchant-service is a **leaf service**: other services call it, it doesn't call them.

4. Code Structure

```
merchant-service/src/main/java/com/acquirerx/merchant/
├─ MerchantServiceApplication.java
├─ merchant/
│   ├─ controller/
│   │   ├─ MerchantController.java
│   │   └─ MerchantOnboardingController.java
│   ├─ dto/ (Merchant, KYC, Pricing, Settlement DTOs)
│   ├─ entity/
│   │   ├─ Merchant.java
│   │   ├─ MerchantKYC.java
│   │   ├─ PricingModel.java
│   │   └─ SettlementProfile.java
│   └─ service/
│       ├─ MerchantService.java
│       └─ MerchantOnboardingService.java
└─ store/
    ├─ controller/StoreController.java
    ├─ dto/ (Store DTOs, Stats DTO)
    ├─ entity/Store.java
    └─ service/StoreService.java
```

Why split into `merchant/` and `store/`: Even though both live in the same service and database, they're conceptually distinct. Stores are owned by merchants, but their lifecycle and

operations differ enough to warrant separate packages. This makes code navigation easy and keeps DTOs scoped.

5. Entities

5.1 `Merchant` (table: `merchant`)

Field	Type	Purpose
<code>merchantId</code>	Long PK	Auto-increment
<code>legalName</code>	String	Registered legal name (e.g., "TechMart Retail Pvt Ltd")
<code>doingBusinessAs</code>	String	Trade name (e.g., "TechMart")
<code>mcc</code>	String	Merchant Category Code — 4 digits, e.g., <code>5411</code> (grocery), <code>5812</code> (restaurant)
<code>contactInfo</code>	String	Phone or email
<code>riskLevel</code>	String	LOW / MEDIUM / HIGH — set by risk team during onboarding
<code>status</code>	String	PENDING (newly created) → ACTIVE (after KYC done) → INACTIVE (suspended)
<code>createdAt</code>	LocalDateTime	Auto-set

Why MCC matters: The Merchant Category Code is a 4-digit ISO standard. It drives:

- Fee calculation (different MCCs have different interchange rates)
- Risk decisioning (e.g., MCC 7995 = gambling = higher scrutiny)
- Reporting (volume by industry)

Why status starts at PENDING: A merchant can be created (record exists, has an ID) but not yet activated. Activation requires:

1. KYC documents submitted and verified
2. Pricing model assigned
3. Settlement profile assigned

Only then is the merchant moved to ACTIVE, allowing terminals to be assigned and transactions to flow.

5.2 `Store` (table: `store`)

A merchant can have multiple physical stores. Each store can have multiple terminals.

Field	Type	Purpose
<code>storeId</code>	Long PK	Auto-increment
<code>storeName</code>	String	e.g., "TechMart - MG Road"
<code>address</code>	String	Physical address
<code>region</code>	String	NORTH / SOUTH / EAST / WEST / NA
<code>status</code>	String	ACTIVE / INACTIVE / PENDING
<code>merchantId</code>	Long FK	Reference to merchant (no JPA <code>@ManyToOne</code> — stored as plain Long)
<code>createdAt</code>	LocalDateTime	Auto-set

Why `merchantId` as Long, not `@ManyToOne Merchant`: This is a key microservices design decision. Even though both are in the same DB and you *could* use a JPA relationship, using just an ID:

- Keeps the data model microservice-friendly (works the same way other services reference merchant)
- Avoids unintended lazy-loading bugs
- Makes serialization simpler (no proxy objects in JSON)
- Matches the convention used elsewhere in the codebase

Why `region`: Region is used in fee rule matching (some fee rules apply only to certain regions) and in reporting (volume by region).

5.3 `MerchantKYC` (table: `merchant_kyc`)

Field	Type	Purpose
<code>kycId</code>	Long PK	

Field	Type	Purpose
<code>merchantId</code>	Long FK	
<code>documentType</code>	String	PAN_CARD / GST_CERTIFICATE / BANK_STATEMENT / etc.
<code>documentRef</code>	String	Reference number or storage URL
<code>verifiedDate</code>	LocalDateTime	When the doc was verified (null until verified)
<code>status</code>	String	PENDING → VERIFIED or REJECTED
<code>notes</code>	String	Reviewer comments (especially on rejection)
<code>submittedAt</code>	LocalDateTime	When uploaded

Why a separate table (not columns on Merchant): A merchant submits multiple KYC documents (PAN, GST, bank proof, address proof). Each is reviewed independently. Each has its own status and verification timestamp. Putting them in a child table is the only normalized way to model this.

5.4 `PricingModel` (table: `pricing_model`)

The fee rates negotiated with the merchant.

Field	Type	Purpose
<code>pricingId</code>	Long PK	
<code>merchantId</code>	Long FK	
<code>modelType</code>	String	MDR / IC_PLUS_PLUS / BLEND
<code>mdrPct</code>	Double	Merchant Discount Rate (% of txn)
<code>perTxnFee</code>	Double	Flat per-transaction fee
<code>schemeFeePassThrough</code>	Boolean	Whether to charge scheme fees separately
<code>effectiveFrom</code>	LocalDate	Pricing valid from this date
<code>effectiveTo</code>	LocalDate	Pricing valid until this date (null = indefinite)
<code>status</code>	String	ACTIVE / INACTIVE
<code>createdAt</code>	LocalDateTime	

Why an effective date range: Merchants may renegotiate pricing. Old PricingModel records aren't deleted — they're deactivated and a new one is created with a new `effectiveFrom`. This preserves history (essential for auditing past transactions: "what rate was applied on Sep 5?").

Pricing model types explained:

Type	How it works
MDR (Merchant Discount Rate)	Single percentage of transaction amount. Simple, common for small merchants.
IC++ (Interchange Plus Plus)	Pass-through interchange + scheme fee + acquirer markup. Transparent, common for large merchants.
BLEND	Hybrid — flat rate that bundles everything.

Note: AcquirerX stores these for record-keeping; the actual fee calculation in transaction-service uses `FeeRule` entries (see Deep-Dive 6).

5.5 `SettlementProfile` (table: `settlement_profile`)

How the merchant gets paid.

Field	Type	Purpose
<code>settleProfileId</code>	Long PK	
<code>merchantId</code>	Long FK	
<code>settlementCycle</code>	String	DAILY / T_PLUS_1 / T_PLUS_2 / WEEKLY
<code>bankAccountRef</code>	String	Masked bank reference (never raw account number)
<code>reservePct</code>	Double	% of revenue held back as chargeback reserve (0-100)
<code>status</code>	String	ACTIVE / INACTIVE
<code>createdAt</code> , <code>updatedAt</code>		

Settlement cycle:

- DAILY — paid same day after EOD batch

- T+1 — paid next business day (most common)
- T+2 — paid two days later
- WEEKLY — accumulated over the week, paid weekly

Reserve percentage: For high-risk merchants (e.g., new merchants, high-chargeback industries), the acquirer holds back a percentage of revenue as protection. If chargebacks come in later, they can be deducted from the reserve. Typical reserve: 5-20% for risky merchants, 0% for established ones.

Why bank account is masked: PCI/PII concerns. Storing raw bank numbers is risky. Instead, a tokenized reference (e.g., `BANK_REF_8472`) is stored, and the actual number lives in a vault (or is keyed off a real banking integration in production).

6. The MerchantService

6.1 Responsibility

`MerchantService.java` handles the merchant entity itself — CRUD, search, status transitions, stats.

6.2 Typical method shape

```
java
```

```

@Service
@RequiredArgsConstructor
public class MerchantService {

    private static final Set<String> SORTABLE_FIELDS = Set.of(
        "merchantId", "legalName", "mcc", "riskLevel", "status", "createdAt"
    );

    private final MerchantRepository merchantRepository;

    public MerchantResponseDTO create(MerchantRequestDTO dto) {
        Merchant m = new Merchant();
        m.setLegalName(dto.getLegalName());
        m.setDoingBusinessAs(dto.getDoingBusinessAs());
        m.setMcc(dto.getMcc());
        m.setContactInfo(dto.getContactInfo());
        m.setRiskLevel(dto.getRiskLevel());
        m.setStatus("PENDING"); // always start at PENDING
        return toResponse(merchantRepository.save(m));
    }

    public MerchantResponseDTO updateStatus(Long id, String newStatus) {
        Merchant m = merchantRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Merchant not found: "
                + id));
        validateStatusTransition(m.getStatus(), newStatus);
        m.setStatus(newStatus);
        return toResponse(merchantRepository.save(m));
    }

    public PagedResponseDTO<MerchantResponseDTO> search(MerchantFilterDTO filter,
        PaginationParams pagination) {
        pagination.validateSortField(SORTABLE_FIELDS);
        Page<Merchant> page = merchantRepository.findByFiltersPaged(
            filter.getLegalName(), filter.getMcc(), filter.getStatus(),
            filter.getRiskLevel(), filter.getContactInfo(),
            pagination.toPageable()
        );
        return new PagedResponseDTO<>(page.map(this::toResponse));
    }

    public MerchantStatsDTO getStats() {
        long total = merchantRepository.count();
        long active = merchantRepository.countByStatus("ACTIVE");
    }
}

```



```
        long pending = merchantRepository.countByStatus("PENDING");
        long inactive = merchantRepository.countByStatus("INACTIVE");
        Map<String, Long> byMcc = merchantRepository.countByMcc();
        return new MerchantStatsDTO(total, active, pending, inactive, byMcc);
    }
}
```

Key design patterns:

- `SORTABLE_FIELDS` whitelist — prevents SQL injection and bad sort fields
 - Status always starts as PENDING — onboarding flow is enforced via state machine
 - `validateStatusTransition` — prevents illegal jumps (e.g., PENDING → INACTIVE without going through ACTIVE)
 - DTO mapping in private helper — keeps entity → response logic in one place
 - `PagedResponseDTO` — uniform pagination wrapper across all services
-

7. The MerchantOnboardingService

7.1 Responsibility

Manages the three sub-resources of a merchant: KYC documents, pricing models, settlement profiles. These are kept in a separate service class because:

- Onboarding has its own logic flow (verify → assign pricing → assign settlement → activate)
- Keeps `MerchantService` focused on the merchant entity itself
- Easy to evolve onboarding rules without touching merchant CRUD

7.2 Typical method shape

```
java
```

```

@Service
@RequiredArgsConstructor
public class MerchantOnboardingService {

    private final MerchantRepository merchantRepository;
    private final MerchantKYCRepository kycRepository;
    private final PricingModelRepository pricingRepository;
    private final SettlementProfileRepository settlementRepository;

    // — KYC —————
    public MerchantKYCResponseDTO submitKyc(MerchantKYCRequestDTO dto) {
        // Verify merchant exists
        merchantRepository.findById(dto.getMerchantId())
            .orElseThrow(() -> new ResourceNotFoundException(
                "Merchant not found: " + dto.getMerchantId()));

        MerchantKYC kyc = new MerchantKYC();
        kyc.setMerchantId(dto.getMerchantId());
        kyc.setDocumentType(dto.getDocumentType());
        kyc.setDocumentRef(dto.getDocumentRef());
        kyc.setNotes(dto.getNotes());
        kyc.setStatus("PENDING");
        return toKycResponse(kycRepository.save(kyc));
    }

    public MerchantKYCResponseDTO verifyKyc(Long kycId) {
        MerchantKYC kyc = kycRepository.findById(kycId)
            .orElseThrow(() -> new ResourceNotFoundException("KYC not found"));
        kyc.setStatus("VERIFIED");
        kyc.setVerifiedDate(LocalDate.now());
        return toKycResponse(kycRepository.save(kyc));
    }

    public MerchantKYCResponseDTO rejectKyc(Long kycId, String reason) {
        MerchantKYC kyc = kycRepository.findById(kycId)
            .orElseThrow(() -> new ResourceNotFoundException("KYC not found"));
        kyc.setStatus("REJECTED");
        kyc.setNotes(reason);
        return toKycResponse(kycRepository.save(kyc));
    }

    // — Pricing —————
    public PricingModelResponseDTO createPricing(PricingModelRequestDTO dto) {

```

```

        // Validate no overlapping active pricing for same merchant
        validateNoOverlappingActivePricing(dto.getMerchantId(),
                                           dto.getEffectiveFrom(),
                                           dto.getEffectiveTo());

        PricingModel p = new PricingModel();
        // ... map fields, set status="ACTIVE"
        return toPricingResponse(pricingRepository.save(p));
    }

    public PricingModelResponseDTO deactivatePricing(Long pricingId) {
        PricingModel p = pricingRepository.findById(pricingId)
            .orElseThrow(...);
        p.setStatus("INACTIVE");
        return toPricingResponse(pricingRepository.save(p));
    }

    // — Settlement Profile —————
    // create / update / deactivate / list — similar pattern
}

```

Notable design decisions:

- **Validate merchant exists before creating sub-resource** — prevents orphan KYC/pricing records
- **Reject sets status + notes (rejection reason)** — auditable
- **Verify sets** `verifiedDate` — proves when verification happened
- **Pricing overlap validation** — only one ACTIVE pricing per merchant per time window

8. The StoreService

8.1 Responsibility

Manages stores — CRUD, search, status, stats.

8.2 Typical method shape

```
java
```

```

@Service
@RequiredArgsConstructor
public class StoreService {

    private final StoreRepository storeRepository;
    private final MerchantRepository merchantRepository;

    public StoreResponseDTO createStore(Long merchantId, StoreRequestDTO dto) {
        // Verify merchant is ACTIVE
        Merchant m = merchantRepository.findById(merchantId)
            .orElseThrow(() -> new ResourceNotFoundException("Merchant not found"));
        if (!"ACTIVE".equals(m.getStatus())) {
            throw new IllegalStateException(
                "Cannot create store for non-ACTIVE merchant. Status: " + m.getStatus()
            );
        }

        Store store = new Store();
        store.setStoreName(dto.getStoreName());
        store.setAddress(dto.getAddress());
        store.setRegion(dto.getRegion());
        store.setMerchantId(merchantId);
        store.setStatus("ACTIVE");
        return toStoreResponse(storeRepository.save(store));
    }
    // ... search, getById, updateStatus, getByMerchant, stats
}

```

Why "merchant must be ACTIVE": This is a business rule. You can't operate a store under a merchant whose KYC isn't verified. Enforcing this at the service layer prevents inconsistent state.

9. Repositories

Each entity has a `JpaRepository` interface with custom finders. Example for `MerchantRepository`:

```
java
```

```

public interface MerchantRepository extends JpaRepository<Merchant, Long> {
    long countByStatus(String status);

    @Query("SELECT m.mcc, COUNT(m) FROM Merchant m GROUP BY m.mcc")
    Map<String, Long> countByMcc();

    @Query("SELECT m FROM Merchant m WHERE " +
        "(:legalName IS NULL OR LOWER(m.legalName) LIKE LOWER(CONCAT('%', :legalName, '%') " +
        "(:mcc IS NULL OR m.mcc = :mcc) AND " +
        "(:status IS NULL OR m.status = :status) AND " +
        "(:riskLevel IS NULL OR m.riskLevel = :riskLevel) AND " +
        "(:contactInfo IS NULL OR LOWER(m.contactInfo) LIKE LOWER(CONCAT('%', :contactInfo, '%') " +
    Page<Merchant> findByFiltersPaged(@Param("legalName") String legalName,
                                     @Param("mcc") String mcc,
                                     @Param("status") String status,
                                     @Param("riskLevel") String riskLevel,
                                     @Param("contactInfo") String contactInfo,
                                     Pageable pageable);
}

```

Patterns used:

- `(:param IS NULL OR field = :param)` — optional filter pattern
- `LIKE LOWER(CONCAT('%', :param, '%'))` — case-insensitive partial match
- JPQL aggregates — `GROUP BY` for stats endpoints
- Spring Data method names — `countByStatus`, `findByMerchantId` etc., generated automatically

10. Controllers

10.1 `MerchantController.java` — endpoints

```

GET    /merchants
GET    /merchants/{id}
POST   /merchants
PUT    /merchants/{id}
DELETE /merchants/{id}
POST   /merchants/search

```

```
PATCH  /merchants/{id}/status?newStatus=ACTIVE|INACTIVE|PENDING
GET    /merchants/status/{status}
GET    /merchants/stats
```

10.2 `MerchantOnboardingController.java` — endpoints

```
POST  /merchants/onboarding/kyc
PATCH /merchants/onboarding/kyc/{kycId}/verify
PATCH /merchants/onboarding/kyc/{kycId}/reject?reason=...
GET    /merchants/onboarding/kyc/pending
GET    /merchants/onboarding/kyc/merchant/{merchantId}

POST  /merchants/onboarding/pricing
PATCH /merchants/onboarding/pricing/{pricingId}/deactivate
GET    /merchants/onboarding/pricing/active
GET    /merchants/onboarding/pricing/merchant/{merchantId}

POST  /merchants/onboarding/settlement-profile
PUT    /merchants/onboarding/settlement-profile/{profileId}
PATCH /merchants/onboarding/settlement-profile/{profileId}/deactivate
GET    /merchants/onboarding/settlement-profile/merchant/{merchantId}
```

10.3 `StoreController.java` — endpoints

```
GET    /stores
GET    /stores/{id}
GET    /stores/stats
POST   /stores/search
PATCH /stores/{id}/status?newStatus=...

POST   /merchants/{merchantId}/stores
PUT    /stores/{id}
DELETE /stores/{id}
GET    /merchants/{merchantId}/stores
```

Why this URL structure:

- Creating a store is `POST /merchants/{merchantId}/stores` — store is *under* a merchant, the URL reflects ownership
- Listing all stores is `GET /stores` — global list across merchants

- Editing/deleting is `PUT /stores/{id}` — once a store has an ID, it's addressable directly without going through the merchant

This is **REST resource hierarchy** done correctly: parent-child for creation, flat for read/update of identified resources.

11. Used By (Other Services)

Merchant-service is called by:

Service	When	Endpoint used
transaction-service	During <code>authorize</code> to enrich AuthMessage with merchant MCC/region	<code>GET /merchants/{id}</code>
transaction-service	During <code>createTxnFromAuth</code> if MCC/region missing	<code>GET /merchants/{id}</code>
terminal-service	When validating a merchant before creating terminals	<code>GET /merchants/{id}</code>
settlement-service	During EOD scheduler to fetch all active merchants	<code>GET /merchants/status/ACTIVE</code> (paged)
settlement-service	During settlement to fetch settlement profile	<code>GET /merchants/onboarding/settlement-profile/merchant/{id}</code>
ops-service	When creating disputes (denormalized merchant name)	<code>GET /merchants/{id}</code>

These callers each have their own `MerchantServiceClient` Feign interface plus a fallback class.

12. `application.properties`

properties

```
server.port=9091
spring.application.name=merchant-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.datasource.url=jdbc:mysql://localhost:3306/acquirerx_merchant?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=kail3114
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

server.servlet.context-path=/api/v1
```

13. The Full Onboarding Flow (End-to-End)

1. Admin creates merchant
POST /merchants { legalName, mcc, contactInfo, riskLevel }
→ status = PENDING, merchant ID returned
2. Admin uploads KYC docs (one or more)
POST /merchants/onboarding/kyc { merchantId, documentType, documentRef }
→ kyc status = PENDING
3. KYC reviewer verifies
PATCH /merchants/onboarding/kyc/{kycId}/verify
→ kyc status = VERIFIED
4. Admin assigns pricing
POST /merchants/onboarding/pricing { merchantId, modelType, mdrPct, ... }
→ pricing status = ACTIVE
5. Admin assigns settlement profile
POST /merchants/onboarding/settlement-profile { merchantId, settlementCycle, ... }
→ settlement status = ACTIVE
6. Admin activates merchant
PATCH /merchants/{id}/status?newStatus=ACTIVE
→ merchant status = ACTIVE

7. Admin creates physical stores

```
POST /merchants/{merchantId}/stores { storeName, address, region }
```

→ store status = ACTIVE

(only allowed if merchant is ACTIVE)

8. Now terminals can be assigned to stores (in terminal-service)

9. Now transactions can flow

Throughout the flow, every action is auditable in audit-service.

14. Why This Design Is Robust

- **Status state machine** — PENDING → ACTIVE → INACTIVE (no unauthorized jumps)
- **Sub-resources isolated** — KYC, pricing, settlement each have their own table, audit trail, lifecycle
- **History preserved** — old pricing/settlement profiles are deactivated, not deleted (audit + accurate historical fee calculation)
- **Validation at service layer** — "merchant must be ACTIVE before stores" is enforced in code, not just trusted from the UI
- **No JPA cross-service joins** — clean microservices boundary, all relationships via Long IDs

15. Common Pitfalls

Pitfall	Symptom	Fix
Trying to create stores under a PENDING merchant	400 / IllegalStateException	Activate the merchant first
Two ACTIVE pricing models with overlapping dates	Two rates apply, one wins randomly	Validate non-overlap at create time
Hard delete of merchant with stores	Orphan stores	Use soft delete (status=INACTIVE) instead
KYC verified without checking merchant exists	Orphan KYC record	Always verify FK before insert

Pitfall	Symptom	Fix
Sorting by un-indexed field	Slow queries on large tables	Use <code>SORTABLE_FIELDS</code> whitelist

16. Summary

Aspect	Detail
Purpose	Merchants, stores, KYC, pricing, settlement profiles
Port	9091
DB	<code>acquirerrx_merchant</code> (5 tables)
Tech	Spring Boot, JPA, MySQL, Lombok
Services	<code>MerchantService</code> , <code>MerchantOnboardingService</code> , <code>StoreService</code>
Controllers	3 (Merchant, MerchantOnboarding, Store)
Callers	transaction, terminal, settlement, ops (all via Feign)
Owns	All merchant data — single source of truth
Calls	None (leaf service — no Feign clients of its own)
Status machine	PENDING → ACTIVE → INACTIVE

Next: Terminal Service — terminals, parameter profiles, health monitoring.